MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 6

Operating Systems

Inside the xv6 File System

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment 3

2.2 Part 1 - Read what xv6 shows 3

2.3 Part 2 - mystat 4

2.4 Part 3 - Measure the limits 5

2.5 Part 4 - Recursive find 7

2.6 Stretch - Inode numbers next to matches 8

3 Conclusions **9**

A Submitted source code **10**

A.1 user/mystat.c 10

A.2 user/find.c 10

1 Introduction

Read the xv6 filesystem layout, implement readable stat output, derive its storage limits, and traverse nested directories using a recursive find utility.

The experiments follow the laboratory handout and were performed inside Ubuntu 26.04.1 LTS in VirtualBox. The figures show actual terminal screenshots taken during execution; complete transcripts and the submitted source files accompany the report.

2 Tasks and results

2.1 Environment

All experiments were performed on 23 October 2026 inside **Ubuntu 26.04.1 LTS (ARM64)** running in VirtualBox as user `misa` on `UbuntuVM`. The Linux kernel is `7.0.0-38-generic`. The installed xv6 checkout is `~/xv6-riscv`, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. GCC and the RISC-V compiler are version 15.2.0; QEMU is version 10.2.1. xv6 is booted from this Ubuntu with `make qemu CPUS=1`.

The figures are screenshots of the Ubuntu terminal during execution. Complete command output is provided in `terminal-output.txt`; build logs are in `build-output.txt` for the xv6 labs. PIDs, inode numbers and free-space values below refer to these recorded runs.

This laboratory used its own freshly built xv6 image. The image was rebuilt again before the required nested-find test, so that its setup started without the earlier hard-link experiment. The lab 06 Makefile retains `sleep` and `args` and adds `mystat` and `find`.

2.2 Part 1 - Read what xv6 shows

xv6 ls prints four fields: name, numeric type, inode number and logical size in bytes. It does not print Unix permission bits or ownership fields like Linux ls -l.

2.2.1 1. Meaning of types 1, 2 and 3

`kernel/stat.h` defines **T_DIR = 1** (directory), **T_FILE = 2** (regular file) and **T_DEVICE = 3** (device). The console is a device inode connected to a driver rather than ordinary file data.

2.2.2 2. Root inode and the dot entries

The root inode is **1**, specified by `ROOTINO` in `kernel/fs.h`. Both `.` and `..` in the root name inode 1 because the root is its own parent. The root directory's logical size in this image is 1024 bytes.

2.2.3 3. Empty-directory size

After `mkdir a`, ls a showed only `.` and `..` and a size of **32 bytes**. `struct dirent` contains a two-byte inode number and a fourteen-byte name, making each entry **16 bytes**. Therefore two entries occupy 32 logical bytes, although their storage is allocated in whole blocks. An exactly fourteen-byte name need not

be NUL-terminated on disk.

```
$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
$
```

Figure 2.1 - Empty-directory entries, readable stat output and changes in hard-link counts.

2.3 Part 2 - mystat

`user/mystat.c` accepts one or more paths, calls stat for each, maps numeric types to readable strings and prints the inode number, link count and size. An invalid path reports an error on descriptor 2 and does not suppress later valid paths. Usage/stat errors cause a nonzero exit status.

2.3.1 1. Outputs after ln and after rm

Initially README had inode 2, one link and size 2441. `ln README r2` created a second name for that same inode and raised both reported link counts to 2. Removing README left r2 with inode 2, the same size and contents, and one remaining link. `cat r2 | wc` confirmed that the file remained readable.

The root's link count was 2 at the stat check because directory a had already been created. xv6's initial directory link accounting does not count `.` in the same way as Linux's usual new-directory convention; its initial root count is 1.

2.3.2 2. Where unlink decreases the count and when blocks are freed

`kernel/sysfile.c:244` in the installed checkout contains `ip->nlink--;` inside `sys_unlink`, followed by `iupdate(ip)` and `iunlockput(ip)`. The earlier matching decrement in sys_link is error rollback, not the normal unlink path.

`sys_unlink` clears the directory slot and decreases the inode's link count. In `kernel/fs.c`, iput detects the last in-memory reference to an unlinked inode (`ref == 1`, valid inode, nlink 0). It calls itrunc to release the direct/indirect data blocks and later makes the inode reusable. An open file holds an additional reference, delaying reclamation until it is closed or the process exits. These updates occur within filesystem transactions.

2.3.3 3. Fields of struct dinode

The on-disk inode has `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT + 1]`. The first four fields are short integers, size is a uint, and addrs contains thirteen uint block numbers: twelve direct addresses and one indirect address. The structure occupies 64 bytes.

There is no filename, owner UID/GID, permission mode or timestamp. Device inodes use major/minor to select their driver/device. This xv6 baseline has neither symbolic links nor Unix permission checking. The in-memory inode separately stores synchronization and reference-count information.

2.4 Part 3 - Measure the limits

```
misa@UbuntuVM:xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
misa@UbuntuVM:xv6-riscv$ grep nmeta /home/misa/os-labs-04-08-20261023/evidence/lab06-build.txt
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
misa@UbuntuVM:xv6-riscv$ grep -n "ip->nlink--" kernel/sysfile.c
178:  ip->nlink--;
244:  ip->nlink--;
misa@UbuntuVM:xv6-riscv$ sed -n "27,39p" kernel/fs.h
#define NINDIRECT (BSIZE / sizeof(uint))
#define MAXFILE   (NDIRECT + NINDIRECT)
#define NLINK_MAX 32767 // nlink is a short; refuse links past its maximum

// On-disk inode structure
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Number of links to inode in file system
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses
};
misa@UbuntuVM:xv6-riscv$
```

Figure 2.2 - Installed filesystem constants, actual mkfs layout, unlink locations and inode fields.

2.4.1 1. Long filenames

`mkdir averyveryverylongname` appeared in ls as **averyveryveryl**, the first fourteen characters. The responsible constant is `DIRSIZ = 14`. Distinct long names sharing those initial bytes can resolve to the same directory component; this is a stored-name limit, not display clipping.

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
averyveryveryl 1 29 32
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
mystat         2 25 36456
find           2 26 44496
console        3 27 0
a              1 28 32
r2             2 2 2441
$
```

Figure 2.3 - The long directory name is truncated to the fourteen-byte xv6 limit.

2.4.2 2. Maximum file size

`BSIZE = 1024`, `NDIRECT = 12`, and `NINDIRECT = BSIZE / sizeof(uint) = 1024 / 4 = 256`. Therefore the maximum content is **(12 + 256) * 1024 = 274432 bytes = 268 KiB**.

The indirect block stores block numbers, so it adds addressing capacity rather than an extra data block in the content calculation. A maximum-size file additionally occupies that indirect metadata block, an inode and other filesystem metadata. Available free space can impose a lower practical limit than this per-file addressing bound.

2.4.3 3. Disk layout from the actual nmeta line

The build printed `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000`.

Region	Blocks	Indices
Boot block and superblock	2	0 and 1
Log header plus 30 payload blocks	31	2 through 32
Inode region	13	33 through 45
Free-block bitmap	1	46
Data region	1953	47 through 1999

The total is **2 + 31 + 13 + 1 + 1953 = 2000 blocks**, or 2,048,000 bytes. The 1953-block data region includes already allocated directories and programs; it is not a count of unused blocks.

2.5 Part 4 - Recursive find

`user/find.c` compares each last pathname component with the requested name using strcmp. It handles matching files and directories, skips free slots (`inum == 0`) and never recurses into `.` or `..`. Each on-disk name is copied into a fifteen-byte NUL-terminated buffer before string operations. Path construction is bounded by `MAXPATH`; trailing slashes on the starting path are normalized.

Every directory descriptor is closed before descending into a child. The implementation reopens the parent and skips to its next entry, trading O(n squared) reads per directory for bounded descriptor use. A shared path buffer reduces recursive stack use. This is sufficient for the small teaching filesystem; directories should remain unchanged while it walks them, and path/stack limits still bound the supported depth.

2.5.1 Fresh-image nested-directory test

A new image was built before this setup:

`mkdir a` was also executed at the start of this fresh-image test. The output contains all three r2 paths, and searching for b also finds the directory a/b. The traversal is not restricted to regular files.

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find . r2 -n
./a/b/r2 inode 30
./a/r2 inode 31
./r2 inode 3
$ mystat cat r2
cat: file, inode 3, links 2, size 36728 bytes
r2: file, inode 3, links 2, size 36728 bytes
$
```

Figure 2.4 - A fresh-image nested find test and the optional inode-number output.

2.5.2 Invalid paths and arguments

The additional checks exercised `mystat missing cat`, `find no-such r2`, `find a/ b`, a bad `-x` option, and missing arguments. mystat continued to print cat after the missing-path error. find reported invalid input explicitly, while the trailing-slash form found the same a/b directory.



Figure 2.5 - Error handling, continued processing of valid paths and trailing-slash normalization.

2.6 Stretch - Inode numbers next to matches

Only `./r2` has inode **3**, shared with cat. `./a/b/r2` and `./a/r2` have independent inodes **30** and **31**. Equal names do not imply a shared file; equal inode numbers on the same filesystem do.

3 Conclusions

The xv6 implementation makes inode identity and directory structure directly visible. mystat confirms that hard links share one inode and that unlink changes the link count rather than copying or immediately deleting data. The disk constants yield a 268 KiB maximum file and a 2000-block image. find locates nested matches and its optional inode display distinguishes equal names from actual hard links.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 6: *Inside the xv6 File System*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/06aad25c735fd3159bdfae5680be4eab7b1668b2`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/mystat.c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

static char *
typename(int type)
{
  switch(type){
  case T_DIR: return "directory";
  case T_FILE: return "file";
  case T_DEVICE: return "device";
  default: return "unknown";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
  int failed = 0;
  if(argc < 2){
    fprintf(2, "usage: mystat <path> [path ...]\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      failed = 1;
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(failed);
}
```

A.2 user/find.c

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "kernel/param.h"
#include "kernel/fcntl.h"
#include "user/user.h"

static char path[MAXPATH];
static char *wanted;
static int show_inode;
static int failed;

/* Read one entry and close before recursion: depth never consumes FDs.
 * Reopening and skipping costs O(n^2) reads per directory, acceptable here.
 * The directory must not be modified during the walk. */
static int
```

```
entry_at(int index, struct dirent *entry)
{
  int fd = open(path, O_RDONLY);
  if(fd < 0){
    fprintf(2, "find: cannot open %s\n", path);
    failed = 1;
    return -1;
  }
  int n = 0;
  for(int i = 0; i <= index; i++){
    n = read(fd, entry, sizeof(*entry));
    if(n != sizeof(*entry))
      break;
  }
  close(fd);
  if(n == 0)
    return 0;
  if(n != sizeof(*entry)){
    fprintf(2, "find: directory read failed: %s\n", path);
    failed = 1;
    return -1;
  }
  return 1;
}

static void
walk(void)
{
  struct stat st;
  struct dirent entry;
  char name[DIRSIZ + 1];
  if(stat(path, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    failed = 1;
    return;
  }
  char *base = path;
  for(char *p = path; *p; p++)
    if(*p == '/' && p[1])
      base = p + 1;
  if(strcmp(base, wanted) == 0){
    if(show_inode)
      printf("%s inode %d\n", path, st.ino);
    else
      printf("%s\n", path);
  }
  if(st.type != T_DIR)
    return;
  int len = strlen(path);
  for(int index = 0; entry_at(index, &entry) > 0; index++){
    if(entry.inum == 0)
      continue;
    memmove(name, entry.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;
    int slash = len > 0 && path[len - 1] != '/';
    if(len + slash + strlen(name) + 1 > sizeof(path)){
      fprintf(2, "find: path too long below %s\n", path);
      failed = 1;
      continue;
    }
    if(slash)
      path[len] = '/';
    strcpy(path + len + slash, name);
    walk();
```

```
    path[len] = 0;
  }
}

int
main(int argc, char *argv[])
{
  if((argc != 3 && argc != 4) ||
     (argc == 4 && strcmp(argv[3], "-n") != 0)){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  if(strlen(argv[1]) == 0 || strlen(argv[1]) >= sizeof(path)){
    fprintf(2, "find: invalid starting path\n");
    exit(1);
  }
  strcpy(path, argv[1]);
  int len = strlen(path);
  while(len > 1 && path[len - 1] == '/')
    path[--len] = 0;
  wanted = argv[2];
  show_inode = argc == 4;
  walk();
  exit(failed);
}
```